configs/phase3.yaml

```yaml
# Phase 1 + Phase 2 + Phase 3 settings (ingestion, speech recognition, diarization).
# Every threshold lives here; nothing is hard-coded in src/.
data_dir: data
results_dir: results
log_dir: logs
seed: 42

ingestion:
  max_duration_s: 300        # development cap; 0 = no limit
  min_duration_s: 1.0        # reject clips shorter than this
  audio_sample_rate: 16000   # Whisper / pyannote expect 16 kHz mono
  force_cfr: true            # re-encode variable frame rate to constant frame rate
  vfr_tolerance: 0.02        # relative gap between r_frame_rate and avg_frame_rate
  random_frame_checks: 20    # number of random seek-and-read checks
  black_frame_mean: 8.0      # mean pixel value (0-255) below which a frame counts as black
  max_bad_frame_fraction: 0.1  # tolerated share of black/unreadable sampled frames
  x264_crf: 18               # quality used only when a re-encode is needed
  x264_preset: veryfast

gates:                       # Phase 1 pass/fail thresholds (Section 5, Phase 1)
  frame_count_rel_diff: 0.01 # OpenCV vs ffprobe frame count, relative
  audio_video_diff_s: 0.1    # |audio duration - video duration|
  decode_errors: 0           # lines printed by the ffmpeg null decode

asr:                         # Phase 2 - Whisper speech recognition
  backend: faster-whisper    # faster-whisper | openai-whisper
  model_size: small          # tiny | base | small | medium | large-v3 (see model-size study)
  language: en               # null = auto-detect
  device: auto               # auto | cpu | cuda
  compute_type: auto         # faster-whisper only: auto | int8 | float16 | float32
  vad_filter: true           # Silero VAD (faster-whisper only): suppresses silence hallucination
  beam_size: 5
  condition_on_previous_text: false  # false reduces repetition loops / runaway hallucination
  # --- filtering ---
  max_no_speech_prob: 0.6    # drop a segment when no_speech_prob > this AND ...
  min_avg_logprob: -1.0      # ... avg_logprob < this (Whisper's own heuristic)
  hallucination_phrases:     # known silence hallucinations (matched on short segments)
    - thanks for watching
    - thank you for watching
    - please subscribe
    - subtitles by
    - amara.org
    - like and subscribe
  # --- balloon-sized segmentation ---
  max_words: 12              # split segments longer than this ...
  max_duration_s: 4.0        # ... or longer than this
  split_gap_s: 0.8           # always split at a pause this long inside a segment
  min_duration_s: 0.5        # merge fragments shorter than this
  merge_gap_s: 0.5           # ... only with a neighbour closer than this
  merge_max_words: 16        # ... and only if the merged line stays this short
  # --- evaluation ---
  timestamp_match_min_ratio: 0.5   # text similarity needed to pair a manual mark with a segment
  nonspeech_overlap_ratio: 0.5     # segment counts as hallucination if this much lies in silence

asr_gates:                   # suggested starting thresholds - tune after your first run
  wer_max: 0.15              # clean dialogue
  start_mae_s_max: 0.5
  hallucination_rate_max: 0.05

diar:                        # Phase 3 - speaker diarization (who spoke when)
  backend: pyannote
  model: pyannote/speaker-diarization-3.1   # gated model: accept its terms on Hugging Face first
  hf_token_env: HF_TOKEN     # NAME of the environment variable that holds your token
  device: auto               # auto | cpu | cuda
  min_speakers: null         # pass the true number if known (null = let the model decide)
  max_speakers: null
  # --- turn smoothing ---
  merge_gap_s: 0.3           # merge same-speaker turns separated by less than this
  min_turn_s: 0.3            # drop turns shorter than this
  # --- Whisper segment <-> speaker alignment ---
  min_overlap_ratio: 0.5     # best speaker must cover at least this share of the segment, else unknown
  mixed_second_ratio: 0.3    # a 2nd speaker covering this share of the segment -> 'mixed'
  # --- evaluation ---
  collar_s: 0.25             # DER forgiveness collar around reference boundaries (pyannote convention)
  der_step_s: 0.01           # frame step of the built-in DER calculator
  use_pyannote_metrics: true # also report DER from pyannote.metrics when it is installed

diar_gates:                  # suggested starting thresholds - film audio with music is hard
  der_max: 0.25
  attribution_acc_min: 0.80
```

src/audio/diar_postprocess.py

```python
"""Turn raw diarization turns into clean turns and attach a speaker to every Whisper segment.

Pure functions (no model, no files) so they are easy to unit-test.

Turn format   : {"start": float, "end": float, "speaker": str}
Segment labels: a real speaker name, or the special labels MIXED / UNKNOWN.
"""
from __future__ import annotations

MIXED = "mixed"
UNKNOWN = "unknown"
SPECIAL = (MIXED, UNKNOWN)


def overlap(a0: float, a1: float, b0: float, b1: float) -> float:
    """Length (seconds) of the intersection of two intervals; 0 if they do not touch."""
    return max(0.0, min(a1, b1) - max(a0, b0))


# ---- 1. smoothing -----------------------------------------------------------
def smooth_turns(turns: list[dict], dcfg: dict) -> list[dict]:
    """Merge same-speaker turns closer than merge_gap_s, then drop turns shorter than min_turn_s."""
    by_spk: dict[str, list[dict]] = {}
    for t in turns:
        if t["end"] > t["start"]:                       # ignore zero / negative-length turns
            by_spk.setdefault(t["speaker"], []).append(dict(t))
    merged: list[dict] = []
    for spk, items in by_spk.items():
        items.sort(key=lambda t: (t["start"], t["end"]))
        cur = items[0]
        for t in items[1:]:
            if t["start"] - cur["end"] < dcfg["merge_gap_s"]:
                cur["end"] = max(cur["end"], t["end"])
            else:
                merged.append(cur)
                cur = t
        merged.append(cur)
    kept = [t for t in merged if t["end"] - t["start"] >= dcfg["min_turn_s"]]
    kept.sort(key=lambda t: (t["start"], t["end"], t["speaker"]))
    return [{"start": round(t["start"], 3), "end": round(t["end"], 3), "speaker": t["speaker"]}
            for t in kept]


def relabel_by_first_appearance(turns: list[dict]) -> list[dict]:
    """Rename speakers SPEAKER_00, SPEAKER_01 ... in order of first appearance (deterministic)."""
    mapping: dict[str, str] = {}
    out = []
    for t in sorted(turns, key=lambda t: (t["start"], t["end"], t["speaker"])):
        if t["speaker"] not in mapping:
            mapping[t["speaker"]] = f"SPEAKER_{len(mapping):02d}"
        out.append(dict(t, speaker=mapping[t["speaker"]]))
    return out


# ---- 2. alignment with Whisper segments -------------------------------------
def attribute_segments(segments: list[dict], turns: list[dict], dcfg: dict) -> list[dict]:
    """For every segment choose the speaker with the largest time overlap.

    Label rules (a wrong speaker is worse than none, so we abstain):
      no turn overlaps the segment                    -> unknown
      a 2nd speaker covers >= mixed_second_ratio of it  -> mixed
      best speaker covers  <  min_overlap_ratio of it   -> unknown
      otherwise                                       -> best speaker
    """
    out = []
    for seg in segments:
        dur = max(seg["end"] - seg["start"], 1e-6)
        per: dict[str, float] = {}
        for t in turns:
            ov = overlap(seg["start"], seg["end"], t["start"], t["end"])
            if ov > 0:
                per[t["speaker"]] = per.get(t["speaker"], 0.0) + ov
        ranked = sorted(per.items(), key=lambda kv: (-kv[1], kv[0]))
        best, best_ov = ranked[0] if ranked else (None, 0.0)
        second_ratio = ranked[1][1] / dur if len(ranked) > 1 else 0.0
        ratio = min(best_ov / dur, 1.0)
        if best is None:
            label, reason = UNKNOWN, "no_speaker_turn"
        elif second_ratio >= dcfg["mixed_second_ratio"]:
            label, reason = MIXED, "overlapping_speakers"
        elif ratio < dcfg["min_overlap_ratio"]:
```

```
            label, reason = UNKNOWN, "low_overlap"
        else:
            label, reason = best, "ok"
        out.append({"segment_id": seg["segment_id"], "speaker": label,
                    "overlap_ratio": round(ratio, 4), "second_ratio": round(min(second_ratio, 1.0), 4),
                    "best_speaker": best, "reason": reason})
    return out


# ---- 3. validation ---------------------------------------------------------
def validate_diarization(turns: list[dict], segment_speakers: list[dict],
                         segments: list[dict]) -> list[str]:
    """Return a list of problems (empty list = OK). Used by the stage gate and tests."""
    problems = []
    for i, t in enumerate(turns):
        if t["end"] <= t["start"]:
            problems.append(f"turn {i}: non-positive duration")
    if len(segment_speakers) != len(segments):
        problems.append(f"{len(segment_speakers)} attributions for {len(segments)} segments")
    known = {t["speaker"] for t in turns}
    ids = [s["segment_id"] for s in segments]
    for ss, sid in zip(segment_speakers, ids):
        if ss["segment_id"] != sid:
            problems.append(f"segment {sid}: attribution out of order")
        if ss["speaker"] not in known and ss["speaker"] not in SPECIAL:
            problems.append(f"segment {sid}: speaker '{ss['speaker']}' not in turns")
        if not 0.0 <= ss["overlap_ratio"] <= 1.0:
            problems.append(f"segment {sid}: overlap_ratio outside [0,1]")
    return problems
```

src/audio/rttm_io.py

```
"""Read and write RTTM files (the standard diarization format).

Line: SPEAKER <file-id> 1 <start> <duration> <NA> <NA> <speaker> <NA> <NA>
"""
from __future__ import annotations

from pathlib import Path


def write_rttm(turns: list[dict], video_id: str, path: str | Path) -> None:
    lines = [f"SPEAKER {video_id} 1 {t['start']:.3f} {t['end'] - t['start']:.3f} "
             f"<NA> <NA> {t['speaker']} <NA> <NA>" for t in turns]
    Path(path).write_text("\n".join(lines) + ("\n" if lines else ""), encoding="utf-8")


def read_rttm(path: str | Path) -> list[dict]:
    turns = []
    for line in Path(path).read_text(encoding="utf-8").splitlines():
        parts = line.split()
        if len(parts) < 8 or parts[0] != "SPEAKER":
            continue
        start, dur = float(parts[3]), float(parts[4])
        turns.append({"start": start, "end": start + dur, "speaker": parts[7]})
    return turns
```

src/audio/diarization.py

```
"""Phase 3: speaker diarization -> speaker turns + an anonymous speaker for every Whisper segment.

Reads  : data/metadata/{id}.json (Phase 1), the 16 kHz WAV, data/subtitles/{id}.json (Phase 2)
Writes : data/diarization/{id}.json and data/diarization/{id}.rttm
Caches : data/diarization/{id}.raw.json  (raw model turns; the slow part is not repeated when
         only the smoothing / alignment settings change)
Metrics: data/diarization/{id}.metrics.json
Speaker names (SPEAKER_00 ...) are arbitrary; Phase 7 maps them to characters.
"""
from __future__ import annotations

import json
import os
import time
from datetime import datetime
from pathlib import Path

from src.audio.diar_postprocess import (MIXED, UNKNOWN, attribute_segments,
                                        relabel_by_first_appearance, smooth_turns,
                                        validate_diarization)
from src.audio.rttm_io import read_rttm, write_rttm
from src.common.base_stage import BaseStage
from src.common.utils import artifact_path, config_hash
from src.evaluation.diarization_metrics import (compute_der, der_pyannote,
```

```
                                            segment_attribution)

RAW_KEYS = ["backend", "model", "device", "min_speakers", "max_speakers"]


class DiarError(RuntimeError):
    pass


# ---- backends ---------------------------------------------------------------
# A backend is any function (wav_path, dcfg, log) -> list of {"start","end","speaker"}.
_MODEL_CACHE: dict = {}


def _diarize_pyannote(wav: str, dcfg: dict, log=None) -> list[dict]:
    try:
        from pyannote.audio import Pipeline
    except ImportError as exc:
        raise DiarError("pyannote.audio is not installed (pip install pyannote.audio)") from exc
    key = ("pa", dcfg["model"], dcfg["device"])
    if key not in _MODEL_CACHE:
        token = os.environ.get(dcfg["hf_token_env"])
        if not token:
            raise DiarError(
                f"environment variable {dcfg['hf_token_env']} is empty. Create a Hugging Face "
                f"token, accept the terms of {dcfg['model']} (and pyannote/segmentation-3.0) "
                "on huggingface.co, then set the variable before running.")
        if log:
            log.info("loading diarization pipeline '%s'", dcfg["model"])
        try:
            try:
                pipe = Pipeline.from_pretrained(dcfg["model"], use_auth_token=token)  # 3.x
            except TypeError:
                pipe = Pipeline.from_pretrained(dcfg["model"], token=token)           # 4.x
        except Exception as exc:  # noqa: BLE001
            raise DiarError(f"could not load '{dcfg['model']}': {type(exc).__name__}: "
                            f"{str(exc)[:200]}") from exc
        if pipe is None:
            raise DiarError(f"'{dcfg['model']}' returned None: token invalid or model terms "
                            "not accepted on huggingface.co")
        import torch
        dev = dcfg["device"]
        if dev == "auto":
            dev = "cuda" if torch.cuda.is_available() else "cpu"
        pipe.to(torch.device(dev))
        _MODEL_CACHE[key] = pipe
    kwargs = {k: dcfg[k] for k in ("min_speakers", "max_speakers") if dcfg.get(k) is not None}
    out = _MODEL_CACHE[key](wav, **kwargs)
    annotation = getattr(out, "speaker_diarization", out)  # 4.x wraps the Annotation
    return [{"start": float(seg.start), "end": float(seg.end), "speaker": str(spk)}
            for seg, _, spk in annotation.itertracks(yield_label=True)]


BACKENDS = {"pyannote": _diarize_pyannote}


def raw_key(dcfg: dict, audio_duration: float) -> str:
    """Hash of everything that changes the raw model output."""
    return config_hash({**{k: dcfg.get(k) for k in RAW_KEYS}, "audio_duration_s": audio_duration})


def run_diarization(dcfg: dict, wav_path: str, log=None, diarize_fn=None):
    """Run the model. Returns (raw_turns, runtime_s)."""
    fn = diarize_fn or BACKENDS.get(dcfg["backend"])
    if fn is None:
        raise DiarError(f"Unknown diar.backend '{dcfg['backend']}' (use {list(BACKENDS)})")
    t0 = time.time()
    turns = fn(str(wav_path), dcfg, log)
    return turns, time.time() - t0


# ---- stage ------------------------------------------------------------------
class DiarStage(BaseStage):
    name = "diarization"
    requires = ["ingestion", "asr"]
    config_keys = ["diar", "diar_gates", "asr", "seed"]

    def __init__(self, cfg, video_id, logger=None, diarize_fn=None):
        super().__init__(cfg, video_id, logger)
        self.diarize_fn = diarize_fn
        self._artifact = artifact_path(cfg, "diarization", video_id, "json")
        self.rttm_path = artifact_path(cfg, "diarization", video_id, "rttm")
        self.raw_path = artifact_path(cfg, "diarization", video_id, "raw.json")
```

```
        self.ref_rttm_path = artifact_path(cfg, "diarization", video_id, "ref.rttm")
        self.gt_path = artifact_path(cfg, "diarization", video_id, "gt.json")

    @property
    def artifact(self) -> Path:
        return self._artifact

    def _load(self, kind: str, hint: str) -> dict:
        path = artifact_path(self.cfg, kind, self.video_id, "json")
        if not path.exists():
            raise DiarError(f"{path} not found: {hint}")
        return json.loads(path.read_text(encoding="utf-8"))

    def _raw_turns(self, meta: dict) -> tuple[list[dict], float, bool]:
        """Raw model turns, from the cache when the model settings did not change."""
        dcfg = self.cfg["diar"]
        key = raw_key(dcfg, meta.get("audio_duration_s") or 0.0)
        if self.raw_path.exists():
            try:
                saved = json.loads(self.raw_path.read_text(encoding="utf-8"))
                if saved.get("raw_cfg") == key:
                    if self.log:
                        self.log.info("[diarization] reusing cached raw turns")
                    return saved["turns"], saved.get("runtime_s", 0.0), True
            except json.JSONDecodeError:
                pass
        turns, runtime = run_diarization(dcfg, meta["audio_path"], self.log, self.diarize_fn)
        self.raw_path.write_text(json.dumps({"raw_cfg": key, "runtime_s": round(runtime, 3),
                                             "turns": turns}, indent=2), encoding="utf-8")
        return turns, runtime, False

    def run(self) -> None:
        meta = self._load("metadata", "run Phase 1 (ingestion) first")
        transcript = self._load("subtitles", "run Phase 2 (asr) first")
        dcfg = self.cfg["diar"]
        segments = transcript["segments"]

        if not meta["has_audio"] or not segments:
            if self.log:
                self.log.info("no audio or no dialogue: writing empty diarization")
            raw, runtime, reused = [], 0.0, False
        else:
            raw, runtime, reused = self._raw_turns(meta)

        turns = relabel_by_first_appearance(smooth_turns(raw, dcfg))
        seg_speakers = attribute_segments(segments, turns, dcfg)
        payload = {
            "video_id": self.video_id,
            "model": dcfg["model"],
            "n_speakers": len({t["speaker"] for t in turns}),
            "n_raw_turns": len(raw),
            "raw_reused": reused,
            "runtime_s": round(runtime, 3),
            "turns": turns,
            "segment_speakers": seg_speakers,
            "created": datetime.now().isoformat(timespec="seconds"),
        }
        self.artifact.write_text(json.dumps(payload, indent=2, ensure_ascii=False),
                                 encoding="utf-8")
        write_rttm(turns, self.video_id, self.rttm_path)

    def evaluate(self, gt=None) -> dict:
        data = json.loads(self.artifact.read_text(encoding="utf-8"))
        transcript = self._load("subtitles", "run Phase 2 (asr) first")
        segs, ss = transcript["segments"], data["segment_speakers"]
        n = len(ss)
        metrics = {
            "model": data["model"],
            "n_turns": len(data["turns"]),
            "n_speakers": data["n_speakers"],
            "n_segments": n,
            "unknown_rate": round(sum(s["speaker"] == UNKNOWN for s in ss) / n, 4) if n else 0.0,
            "mixed_rate": round(sum(s["speaker"] == MIXED for s in ss) / n, 4) if n else 0.0,
            "problems": validate_diarization(data["turns"], ss, segs),
        }
        if gt is None and self.gt_path.exists():
            gt = json.loads(self.gt_path.read_text(encoding="utf-8"))
        gt = gt or {}
        has_ref = self.ref_rttm_path.exists()
        metrics["ground_truth"] = bool(gt) or has_ref
        dcfg = self.cfg["diar"]
        if has_ref:
            ref = read_rttm(self.ref_rttm_path)
```

```
            metrics.update(compute_der(ref, data["turns"], dcfg["collar_s"], dcfg["der_step_s"]))
            if dcfg["use_pyannote_metrics"]:
                metrics["der_pyannote"] = der_pyannote(ref, data["turns"], dcfg["collar_s"])
            metrics["speaker_count_error"] = abs(
                data["n_speakers"] - len({t["speaker"] for t in ref}))
        elif gt.get("n_speakers") is not None:
            metrics["speaker_count_error"] = abs(data["n_speakers"] - gt["n_speakers"])
        if gt.get("segment_labels"):
            metrics.update(segment_attribution(ss, gt["segment_labels"]))
        return metrics

    def gate(self, metrics: dict) -> tuple[bool, str]:
        g = self.cfg["diar_gates"]
        fails = list(metrics["problems"][:3])
        if metrics.get("der") is not None and metrics["der"] > g["der_max"]:
            fails.append(f"DER {metrics['der']:.3f} > {g['der_max']}")
        acc = metrics.get("segment_accuracy")
        if acc is not None and acc < g["attribution_acc_min"]:
            fails.append(f"segment attribution accuracy {acc:.3f} < {g['attribution_acc_min']}")
        if fails:
            return False, "; ".join(fails)
        if not metrics["ground_truth"]:
            return True, "structure OK; no reference labels (DER / accuracy not evaluated)"
        return True, "all checks passed"
```

src/evaluation/diarization_metrics.py

```
"""Phase 3 evaluation: DER, segment attribution accuracy, speaker-count error.

Ground truth (data/diarization/):
  {id}.ref.rttm  hand-made RTTM (Audacity / ELAN), about 10 minutes
  {id}.gt.json   {"segment_labels": {"1": "anna", "2": "ben", ...}, "n_speakers": 2}
Reference and hypothesis speaker names never have to match: both metrics use the optimal
one-to-one label mapping (Hungarian algorithm), so label permutations do not change results.
"""
from __future__ import annotations

import math

import numpy as np
from scipy.optimize import linear_sum_assignment

SPECIAL = ("mixed", "unknown")


# ---- DER ---------------------------------------------------------------------------
def _activity(turns: list[dict], names: list[str], n: int, step: float) -> np.ndarray:
    idx = {s: i for i, s in enumerate(names)}
    mat = np.zeros((len(names), n), dtype=bool)
    for t in turns:
        a, b = int(round(t["start"] / step)), int(round(t["end"] / step))
        mat[idx[t["speaker"]], max(a, 0):max(b, 0)] = True
    return mat


def compute_der(ref: list[dict], hyp: list[dict], collar: float = 0.25,
                step: float = 0.01) -> dict:
    """Frame-based Diarization Error Rate = (missed + false alarm + confusion) / reference speech.

    Frames within collar/2 of every reference turn boundary are ignored (pyannote convention).
    Overlapping speech is counted per speaker. Hypothesis labels are mapped to reference
    labels with the Hungarian algorithm.
    """
    if not ref:
        raise ValueError("reference has no speech turns")
    end = max(t["end"] for t in ref + hyp)
    n = int(math.ceil(end / step)) + 1
    ref_names = sorted({t["speaker"] for t in ref})
    hyp_names = sorted({t["speaker"] for t in hyp})
    R = _activity(ref, ref_names, n, step)
    H = _activity(hyp, hyp_names, n, step)
    keep = np.ones(n, dtype=bool)
    if collar > 0:
        half = collar / 2.0
        for t in ref:
            for b in (t["start"], t["end"]):
                keep[max(int(round((b - half) / step)), 0):max(int(round((b + half) / step)), 0)] = False
    R, H = R[:, keep], H[:, keep]
    n_ref, n_hyp = R.sum(axis=0), H.sum(axis=0)
    total = float(n_ref.sum())
    if total == 0:
        raise ValueError("no reference speech left after applying the collar")
    co = R.astype(np.int64) @ H.astype(np.int64).T          # frames shared by (ref, hyp) pair
```

```
    correct, mapping = 0, {}
    if co.size:
        rows, cols = linear_sum_assignment(-co)
        for r, c in zip(rows, cols):
            correct += int(co[r, c])
            mapping[hyp_names[c]] = ref_names[r]
    missed = float(np.maximum(n_ref - n_hyp, 0).sum())
    false_alarm = float(np.maximum(n_hyp - n_ref, 0).sum())
    confusion = float(np.minimum(n_ref, n_hyp).sum()) - correct
    return {"der": round((missed + false_alarm + confusion) / total, 4),
            "der_missed": round(missed / total, 4),
            "der_false_alarm": round(false_alarm / total, 4),
            "der_confusion": round(confusion / total, 4),
            "ref_speech_s": round(total * step, 2),
            "speaker_mapping": mapping}


def der_pyannote(ref: list[dict], hyp: list[dict], collar: float = 0.25):
    """DER from pyannote.metrics (cross-check). Returns None when it is not installed."""
    try:
        from pyannote.core import Annotation, Segment
        from pyannote.metrics.diarization import DiarizationErrorRate
    except ImportError:
        return None

    def to_ann(turns):
        ann = Annotation()
        for t in turns:
            ann[Segment(t["start"], t["end"])] = t["speaker"]
        return ann
    return round(float(DiarizationErrorRate(collar=collar, skip_overlap=False)(
        to_ann(ref), to_ann(hyp))), 4)


# ---- segment attribution ---------------------------------------------------------
def segment_attribution(segment_speakers: list[dict], ref_labels: dict) -> dict:
    """Accuracy of the speaker given to each Whisper segment, after optimal label mapping.

    segment_accuracy : correct / labelled segments (mixed / unknown count as wrong)
    accuracy_assigned: correct / segments that received a real speaker
    coverage         : assigned / labelled segments
    """
    pairs = [(s["speaker"], ref_labels[str(s["segment_id"])])
             for s in segment_speakers if str(s["segment_id"]) in ref_labels]
    if not pairs:
        return {"segment_accuracy": None, "accuracy_assigned": None, "coverage": None,
                "segments_labelled": 0}
    assigned = [(h, r) for h, r in pairs if h not in SPECIAL]
    hyp_names = sorted({h for h, _ in assigned})
    ref_names = sorted({r for _, r in pairs})
    correct = 0
    if hyp_names:
        co = np.zeros((len(ref_names), len(hyp_names)), dtype=np.int64)
        for h, r in assigned:
            co[ref_names.index(r), hyp_names.index(h)] += 1
        rows, cols = linear_sum_assignment(-co)
        correct = int(co[rows, cols].sum())
    return {"segment_accuracy": round(correct / len(pairs), 4),
            "accuracy_assigned": round(correct / len(assigned), 4) if assigned else None,
            "coverage": round(len(assigned) / len(pairs), 4),
            "segments_labelled": len(pairs)}
```

tests/test_phase3.py

```
"""Phase 3 unit tests. A fake backend is used, so no pyannote model or Hugging Face token is needed.

The stage tests write the Phase 1 metadata and the Phase 2 transcript by hand, so they run
without ffmpeg or Whisper.
"""
from __future__ import annotations

import json
from pathlib import Path

import pytest

from src.audio import diar_postprocess as dp
from src.audio.diarization import DiarError, DiarStage, run_diarization
from src.audio.rttm_io import read_rttm, write_rttm
from src.common.utils import ConfigError, load_config
from src.evaluation import diarization_metrics as dm

ROOT = Path(__file__).resolve().parents[1]
```

```python


# ---- helpers ----------------------------------------------------------------------
def turn(a, b, spk):
    return {"start": a, "end": b, "speaker": spk}


def seg(i, a, b, text="x"):
    return {"segment_id": i, "start": a, "end": b, "text": text}


@pytest.fixture
def dcfg():
    return load_config(ROOT / "configs" / "phase3.yaml")["diar"]


@pytest.fixture
def cfg3(tmp_path):
    c = load_config(ROOT / "configs" / "phase3.yaml")
    c["data_dir"] = str(tmp_path / "data")
    c["results_dir"] = str(tmp_path / "results")
    c["log_dir"] = str(tmp_path / "logs")
    return c


RAW = [turn(0.0, 4.0, "A"), turn(4.5, 9.0, "B"), turn(9.2, 12.0, "A")]
SEGS = [seg(1, 0.2, 3.8), seg(2, 4.6, 8.8), seg(3, 9.3, 11.9)]


def prepare(cfg, vid="d001", segments=None, has_audio=True):
    """Write what Phases 1 and 2 would have written."""
    d = Path(cfg["data_dir"])
    (d / "metadata").mkdir(parents=True, exist_ok=True)
    (d / "subtitles").mkdir(parents=True, exist_ok=True)
    (d / "audio").mkdir(parents=True, exist_ok=True)
    wav = d / "audio" / f"{vid}.wav"
    wav.write_bytes(b"RIFF")
    (d / "metadata" / f"{vid}.json").write_text(json.dumps(
        {"video_id": vid, "has_audio": has_audio, "audio_path": str(wav),
         "audio_duration_s": 12.0}))
    (d / "subtitles" / f"{vid}.json").write_text(json.dumps(
        {"video_id": vid, "segments": SEGS if segments is None else segments}))
    return vid


def fake_backend(turns, calls=None):
    def fn(wav, dcfg, log=None):
        if calls is not None:
            calls.append(1)
        return [dict(t) for t in turns]
    return fn


# ---- config -----------------------------------------------------------------------
def test_phase3_config_loads(dcfg):
    assert dcfg["merge_gap_s"] == 0.3 and dcfg["min_overlap_ratio"] == 0.5


def test_unknown_diar_key_rejected(tmp_path):
    bad = tmp_path / "bad.yaml"
    bad.write_text("diar:\n  not_a_key: 1\n")
    with pytest.raises(ConfigError):
        load_config(bad)


# ---- smoothing --------------------------------------------------------------------
def test_same_speaker_close_turns_are_merged(dcfg):
    out = dp.smooth_turns([turn(0, 2, "A"), turn(2.2, 4, "A")], dcfg)       # gap 0.2 < 0.3
    assert out == [turn(0, 4, "A")]


def test_same_speaker_far_turns_stay_separate(dcfg):
    assert len(dp.smooth_turns([turn(0, 2, "A"), turn(2.5, 4, "A")], dcfg)) == 2


def test_different_speakers_are_never_merged(dcfg):
    assert len(dp.smooth_turns([turn(0, 2, "A"), turn(2.1, 4, "B")], dcfg)) == 2


def test_short_turns_dropped_and_bad_turns_ignored(dcfg):
    out = dp.smooth_turns([turn(0, 0.2, "A"), turn(1, 3, "B"), turn(5, 5, "C"), turn(7, 6, "C")], dcfg)
    assert out == [turn(1, 3, "B")]
    assert all(t["end"] > t["start"] for t in out)
```

```python


def test_relabel_by_first_appearance():
    out = dp.relabel_by_first_appearance([turn(5, 6, "z9"), turn(0, 1, "q1"), turn(2, 3, "z9")])
    assert [t["speaker"] for t in out] == ["SPEAKER_00", "SPEAKER_01", "SPEAKER_01"]


# ---- alignment -------------------------------------------------------------
def test_segment_gets_speaker_with_largest_overlap(dcfg):
    out = dp.attribute_segments([seg(1, 0, 4)], [turn(0, 3.6, "A"), turn(3.8, 4, "B")], dcfg)
    assert out[0]["speaker"] == "A" and out[0]["overlap_ratio"] == pytest.approx(0.9, abs=1e-3)


def test_low_overlap_is_unknown(dcfg):
    out = dp.attribute_segments([seg(1, 0, 10)], [turn(0, 3, "A")], dcfg)
    assert out[0]["speaker"] == dp.UNKNOWN and out[0]["reason"] == "low_overlap"


def test_two_strong_speakers_is_mixed(dcfg):
    out = dp.attribute_segments([seg(1, 0, 4)], [turn(0, 2.2, "A"), turn(1.8, 4, "B")], dcfg)
    assert out[0]["speaker"] == dp.MIXED and out[0]["best_speaker"] in ("A", "B")


def test_no_turn_is_unknown(dcfg):
    out = dp.attribute_segments([seg(1, 20, 22)], [turn(0, 3, "A")], dcfg)
    assert out[0]["speaker"] == dp.UNKNOWN and out[0]["overlap_ratio"] == 0.0


def test_every_segment_gets_exactly_one_label(dcfg):
    turns = [turn(0, 4, "A"), turn(3, 8, "B")]
    segs = [seg(i, i * 1.5, i * 1.5 + 1.2) for i in range(1, 8)]
    out = dp.attribute_segments(segs, turns, dcfg)
    assert len(out) == len(segs)
    assert all(o["speaker"] in {"A", "B", dp.MIXED, dp.UNKNOWN} for o in out)
    assert dp.validate_diarization(turns, out, segs) == []


def test_validate_detects_problems():
    segs = [seg(1, 0, 1)]
    bad = [{"segment_id": 1, "speaker": "GHOST", "overlap_ratio": 1.5}]
    problems = dp.validate_diarization([turn(2, 1, "A")], bad, segs)
    assert len(problems) == 3


# ---- DER -------------------------------------------------------------------
def test_der_perfect_and_label_permutation_invariant():
    ref = [turn(0, 5, "A"), turn(5, 10, "B")]
    assert dm.compute_der(ref, ref, collar=0)["der"] == 0.0
    swapped = [turn(0, 5, "X"), turn(5, 10, "Y")]
    swapped_wrong_order = [turn(0, 5, "B"), turn(5, 10, "A")]      # names clash with ref names
    assert dm.compute_der(ref, swapped, collar=0)["der"] == 0.0
    assert dm.compute_der(ref, swapped_wrong_order, collar=0)["der"] == 0.0


def test_der_missed_false_alarm_confusion():
    ref = [turn(0, 10, "A")]
    m = dm.compute_der(ref, [turn(0, 8, "A")], collar=0)
    assert m["der"] == pytest.approx(0.2, abs=0.01) and m["der_missed"] == pytest.approx(0.2, abs=0.01)
    m = dm.compute_der(ref, [turn(0, 10, "A"), turn(10, 12, "B")], collar=0)
    assert m["der_false_alarm"] == pytest.approx(0.2, abs=0.01)
    ref2 = [turn(0, 5, "A"), turn(5, 10, "B")]
    m = dm.compute_der(ref2, [turn(0, 10, "S")], collar=0)          # one speaker for two
    assert m["der"] == pytest.approx(0.5, abs=0.01) and m["der_confusion"] == pytest.approx(0.5, abs=0.01)


def test_der_collar_forgives_boundary_shift():
    ref = [turn(0, 5, "A"), turn(5, 10, "B")]
    hyp = [turn(0, 5.1, "A"), turn(5.1, 10, "B")]
    assert dm.compute_der(ref, hyp, collar=0)["der"] > 0.0
    assert dm.compute_der(ref, hyp, collar=0.25)["der"] == 0.0


def test_der_empty_reference_raises():
    with pytest.raises(ValueError):
        dm.compute_der([], [turn(0, 1, "A")])


def test_der_empty_hypothesis_is_all_missed():
    m = dm.compute_der([turn(0, 10, "A")], [], collar=0)
    assert m["der"] == pytest.approx(1.0, abs=0.01)


# ---- segment attribution ---------------------------------------------------
```

```python
def test_segment_attribution_uses_optimal_mapping():
    ss = [{"segment_id": 1, "speaker": "SPEAKER_00"}, {"segment_id": 2, "speaker": "SPEAKER_01"},
          {"segment_id": 3, "speaker": "SPEAKER_00"}, {"segment_id": 4, "speaker": "unknown"}]
    ref = {"1": "ben", "2": "anna", "3": "ben", "4": "anna"}       # names differ from hypothesis
    m = dm.segment_attribution(ss, ref)
    assert m["segment_accuracy"] == pytest.approx(0.75) and m["coverage"] == pytest.approx(0.75)
    assert m["accuracy_assigned"] == pytest.approx(1.0)


def test_segment_attribution_without_labels():
    assert dm.segment_attribution([{"segment_id": 1, "speaker": "A"}], {})["segment_accuracy"] is None


# ---- RTTM ------------------------------------------------------------------------
def test_rttm_round_trips(tmp_path):
    turns = [turn(0.0, 4.0, "SPEAKER_00"), turn(4.5, 9.25, "SPEAKER_01")]
    write_rttm(turns, "clip", tmp_path / "x.rttm")
    back = read_rttm(tmp_path / "x.rttm")
    assert [b["speaker"] for b in back] == ["SPEAKER_00", "SPEAKER_01"]
    for b, t in zip(back, turns):
        assert abs(b["start"] - t["start"]) < 1e-3 and abs(b["end"] - t["end"]) < 1e-3


# ---- stage end to end (fake backend) ---------------------------------------------
def test_stage_writes_artifacts_and_passes(cfg3):
    vid = prepare(cfg3)
    stage = DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW))
    assert stage.execute() is True
    data = json.loads(stage.artifact.read_text())
    assert [t["speaker"] for t in data["turns"]] == ["SPEAKER_00", "SPEAKER_01", "SPEAKER_00"]
    assert [s["speaker"] for s in data["segment_speakers"]] == ["SPEAKER_00", "SPEAKER_01", "SPEAKER_00"]
    assert data["n_speakers"] == 2
    assert len(read_rttm(stage.rttm_path)) == 3
    m = json.loads(stage.metrics_path.read_text())
    assert m["passed"] and m["ground_truth"] is False and m["unknown_rate"] == 0.0


def test_gate_passes_with_good_ground_truth(cfg3):
    vid = prepare(cfg3)
    stage = DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW))
    write_rttm([turn(0, 4, "anna"), turn(4.5, 9, "ben"), turn(9.2, 12, "anna")], vid, stage.ref_rttm_path)
    stage.gt_path.write_text(json.dumps({"segment_labels": {"1": "anna", "2": "ben", "3": "anna"},
                                        "n_speakers": 2}))
    assert stage.execute() is True
    m = json.loads(stage.metrics_path.read_text())
    assert m["der"] <= 0.05 and m["segment_accuracy"] == 1.0 and m["speaker_count_error"] == 0


def test_gate_fails_on_bad_diarization(cfg3):
    vid = prepare(cfg3)
    one_speaker = [turn(0.0, 12.0, "A")]                          # model merged both people
    stage = DiarStage(cfg3, vid, diarize_fn=fake_backend(one_speaker))
    write_rttm([turn(0, 4, "anna"), turn(4.5, 9, "ben"), turn(9.2, 12, "anna")], vid, stage.ref_rttm_path)
    stage.gt_path.write_text(json.dumps({"segment_labels": {"1": "anna", "2": "ben", "3": "anna"}}))
    assert stage.execute() is False
    reason = json.loads(stage.metrics_path.read_text())["reason"]
    assert "DER" in reason and "accuracy" in reason


def test_raw_turns_reused_when_only_alignment_settings_change(cfg3):
    vid = prepare(cfg3)
    calls = []
    DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW, calls)).execute()
    DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW, calls)).execute()
    assert len(calls) == 1                                        # whole stage cached
    cfg3["diar"]["min_overlap_ratio"] = 0.8
    DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW, calls)).execute()
    assert len(calls) == 1                                        # stage re-ran, model did not
    cfg3["diar"]["max_speakers"] = 2
    DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW, calls)).execute()
    assert len(calls) == 2                                        # model setting changed


def test_requires_phase1_and_phase2_first(cfg3):
    with pytest.raises(DiarError):
        DiarStage(cfg3, "never_ingested", diarize_fn=fake_backend([])).run()
    prepare(cfg3, "only_meta")
    (Path(cfg3["data_dir"]) / "subtitles" / "only_meta.json").unlink()
    with pytest.raises(DiarError):
        DiarStage(cfg3, "only_meta", diarize_fn=fake_backend([])).run()


def test_no_audio_or_no_dialogue_gives_empty_diarization(cfg3):
```

```
    vid = prepare(cfg3, "d_noaudio", has_audio=False)
    stage = DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW))
    assert stage.execute() is True
    data = json.loads(stage.artifact.read_text())
    assert data["turns"] == [] and data["n_speakers"] == 0
    vid = prepare(cfg3, "d_nodialogue", segments=[])
    assert DiarStage(cfg3, vid, diarize_fn=fake_backend(RAW)).execute() is True


def test_unknown_backend_raises(dcfg):
    with pytest.raises(DiarError):
        run_diarization(dict(dcfg, backend="nope"), "x.wav")
```

main.py

```
"""CineComic-MML entry point.

Usage:
  # Phase 1 only
  python main.py --config configs/phase1.yaml --video clip.mp4 --video-id clip_001
  # Phase 1 + Phase 2
  python main.py --config configs/phase2.yaml --video clip.mp4 --video-id clip_001 \
      --stages ingestion asr
  # Phase 2 only (Phase 1 already done)
  python main.py --config configs/phase2.yaml --video-id clip_001 --stages asr
  # Phase 3 only (needs Phases 1-2 and the HF_TOKEN environment variable)
  python main.py --config configs/phase3.yaml --video-id clip_001 --stages diarization
Later phases will register their stages in STAGES below.
"""
from __future__ import annotations

import argparse
import sys

from src.common.utils import get_logger, load_config, set_seed
from src.ingestion.metadata import IngestionError
from src.ingestion.video_ingestion import ingest

STAGES = ["ingestion", "asr", "diarization"]  # extended phase by phase


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="CineComic-MML pipeline")
    ap.add_argument("--config", required=True, help="YAML config, e.g. configs/phase2.yaml")
    ap.add_argument("--video", default=None, help="video file path or URL (needed for ingestion)")
    ap.add_argument("--video-id", default=None,
                    help="id such as clip_001 (default for ingestion: SHA-256 prefix)")
    ap.add_argument("--stages", nargs="+", default=["ingestion"], choices=STAGES,
                    help="stages to run, in order")
    ap.add_argument("--force", action="store_true", help="ignore the cache and re-run")
    args = ap.parse_args(argv)

    cfg = load_config(args.config)
    set_seed(cfg["seed"])
    video_id = args.video_id

    if "ingestion" in args.stages:
        if not args.video:
            ap.error("--video is required for the ingestion stage")
        try:
            ok, video_id = ingest(cfg, args.video, video_id, force=args.force)
        except IngestionError as exc:
            print(f"INGESTION FAILED: {exc}", file=sys.stderr)
            return 2
        print(f"[ingestion] video_id={video_id} gate={'PASS' if ok else 'FAIL'}")
        if not ok:
            return 1
    elif not video_id:
        ap.error("--video-id is required when ingestion is not in --stages")

    if "asr" in args.stages:
        from src.audio.whisper_asr import AsrError, AsrStage
        log = get_logger(video_id, cfg["log_dir"])
        try:
            ok = AsrStage(cfg, video_id, log).execute(force=args.force)
        except AsrError as exc:
            print(f"ASR FAILED: {exc}", file=sys.stderr)
            return 2
        print(f"[asr] video_id={video_id} gate={'PASS' if ok else 'FAIL'}")
        if not ok:
            return 1

    if "diarization" in args.stages:
```

```
        from src.audio.diarization import DiarError, DiarStage
        log = get_logger(video_id, cfg["log_dir"])
        try:
            ok = DiarStage(cfg, video_id, log).execute(force=args.force)
        except DiarError as exc:
            print(f"DIARIZATION FAILED: {exc}", file=sys.stderr)
            return 2
        print(f"[diarization] video_id={video_id} gate={'PASS' if ok else 'FAIL'}")
        if not ok:
            return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

src/common/utils.py

```
"""Phase 0 shared utilities: config loader, seeding, logging, artifact paths."""
from __future__ import annotations

import hashlib
import json
import logging
import random
from pathlib import Path

import yaml


class ConfigError(ValueError):
    """Raised when a config file contains unknown or invalid keys."""


# Allowed keys. A value of None means "any value"; a dict means "nested section".
# Later phases extend this schema when they add their own config sections.
CONFIG_SCHEMA = {
    "data_dir": None,
    "results_dir": None,
    "log_dir": None,
    "seed": None,
    "ingestion": {
        "max_duration_s": None,
        "min_duration_s": None,
        "audio_sample_rate": None,
        "force_cfr": None,
        "vfr_tolerance": None,
        "random_frame_checks": None,
        "black_frame_mean": None,
        "max_bad_frame_fraction": None,
        "x264_crf": None,
        "x264_preset": None,
    },
    "gates": {
        "frame_count_rel_diff": None,
        "audio_video_diff_s": None,
        "decode_errors": None,
    },
    "asr": {
        "backend": None,
        "model_size": None,
        "language": None,
        "device": None,
        "compute_type": None,
        "vad_filter": None,
        "beam_size": None,
        "condition_on_previous_text": None,
        "max_no_speech_prob": None,
        "min_avg_logprob": None,
        "hallucination_phrases": None,
        "max_words": None,
        "max_duration_s": None,
        "split_gap_s": None,
        "min_duration_s": None,
        "merge_gap_s": None,
        "merge_max_words": None,
        "timestamp_match_min_ratio": None,
        "nonspeech_overlap_ratio": None,
    },
    "asr_gates": {
        "wer_max": None,
        "start_mae_s_max": None,
        "hallucination_rate_max": None,
```

```
    },
    "diar": {
        "backend": None,
        "model": None,
        "hf_token_env": None,
        "device": None,
        "min_speakers": None,
        "max_speakers": None,
        "merge_gap_s": None,
        "min_turn_s": None,
        "min_overlap_ratio": None,
        "mixed_second_ratio": None,
        "collar_s": None,
        "der_step_s": None,
        "use_pyannote_metrics": None,
    },
    "diar_gates": {
        "der_max": None,
        "attribution_acc_min": None,
    },
}


def _validate(cfg: dict, schema: dict, prefix: str = "") -> None:
    for key, value in cfg.items():
        path = f"{prefix}{key}"
        if key not in schema:
            raise ConfigError(f"Unknown config key: '{path}'")
        sub = schema[key]
        if isinstance(sub, dict):
            if not isinstance(value, dict):
                raise ConfigError(f"Config key '{path}' must be a mapping")
            _validate(value, sub, prefix=f"{path}.")


def load_config(path: str | Path, schema: dict | None = None) -> dict:
    """Load a YAML config and reject unknown keys."""
    with open(path, "r", encoding="utf-8") as fh:
        cfg = yaml.safe_load(fh) or {}
    if not isinstance(cfg, dict):
        raise ConfigError("Top level of the config must be a mapping")
    _validate(cfg, schema or CONFIG_SCHEMA)
    return cfg


def config_hash(cfg: dict) -> str:
    """Short stable hash of a config, used for caching and in metrics files."""
    blob = json.dumps(cfg, sort_keys=True, default=str).encode("utf-8")
    return hashlib.sha1(blob).hexdigest()[:8]


def set_seed(seed: int) -> None:
    """Seed Python, NumPy and (if installed) PyTorch."""
    random.seed(seed)
    try:
        import numpy as np
        np.random.seed(seed)
    except ImportError:
        pass
    try:
        import torch
        torch.manual_seed(seed)
    except ImportError:
        pass


def get_logger(video_id: str, log_dir: str | Path = "logs") -> logging.Logger:
    """Logger that writes to the console and to logs/{video_id}.log."""
    logger = logging.getLogger(f"cinecomic.{video_id}")
    if logger.handlers:  # already configured in this process
        return logger
    logger.setLevel(logging.INFO)
    logger.propagate = False
    fmt = logging.Formatter("%(asctime)s | %(levelname)-7s | %(name)s | %(message)s")
    Path(log_dir).mkdir(parents=True, exist_ok=True)
    file_handler = logging.FileHandler(Path(log_dir) / f"{video_id}.log", encoding="utf-8")
    file_handler.setFormatter(fmt)
    console = logging.StreamHandler()
    console.setFormatter(fmt)
    logger.addHandler(file_handler)
    logger.addHandler(console)
    return logger

```

```

def close_logger(logger: logging.Logger) -> None:
    """Release file handles (needed on Windows and in tests)."""
    for handler in list(logger.handlers):
        handler.close()
        logger.removeHandler(handler)


def artifact_path(cfg: dict, kind: str, video_id: str, ext: str = "json") -> Path:
    """Return data/{kind}/{video_id}.{ext} and create the parent folder.

    Example: artifact_path(cfg, "audio", "clip_001", "wav") -> data/audio/clip_001.wav
    """
    path = Path(cfg["data_dir"]) / kind / f"{video_id}.{ext}"
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

requirements.txt

```
# Phase 0/1 dependencies. After your first successful install run
# `pip freeze > requirements.lock.txt` and keep that file for exact reproducibility.
# System tools (not pip): FFmpeg + ffprobe must be on PATH.
PyYAML>=6.0,<7
numpy>=1.26,<3
opencv-python>=4.8,<5
yt-dlp>=2024.1.1
pytest>=8.0,<10
# Phase 2 (speech recognition)
faster-whisper>=1.0      # default backend; or: pip install openai-whisper (backend: openai-whisper)
jiwer>=3.0               # WER / CER
srt>=3.5                 # SRT read/write
# Phase 3 (speaker diarization). Needs a Hugging Face token and accepted model terms.
pyannote.audio>=3.1       # pipeline 'pyannote/speaker-diarization-3.1' (pulls in torch)
pyannote.metrics>=3.2     # DER cross-check (optional)
scipy>=1.11               # Hungarian assignment (linear_sum_assignment)
```

env_check.py

```
"""Phase 0 environment check: imports, versions, CUDA, ffmpeg -> results/environment_report.json."""
from __future__ import annotations

import importlib
import json
import platform
import shutil
import subprocess
import sys
from datetime import datetime
from pathlib import Path

REQUIRED = ["yaml", "numpy", "cv2", "pytest"]  # needed from Phase 1
OPTIONAL = ["torch", "faster_whisper", "whisper", "jiwer", "srt", "yt_dlp", "scenedetect",
            "insightface", "open_clip", "pyannote.audio", "pyannote.metrics",
            "scipy"]


def version_of(name: str):
    try:
        mod = importlib.import_module(name)
        return getattr(mod, "__version__", "ok")
    except Exception as exc:  # noqa: BLE001
        return f"MISSING ({type(exc).__name__})"


def tool_version(tool: str):
    if shutil.which(tool) is None:
        return None
    out = subprocess.run([tool, "-version"], capture_output=True, text=True)
    return out.stdout.splitlines()[0] if out.returncode == 0 and out.stdout else None


def main() -> int:
    report = {"python": platform.python_version(), "platform": platform.platform()}
    failed = []
    for name in REQUIRED:
        report[name] = version_of(name)
        if str(report[name]).startswith("MISSING"):
            failed.append(name)
    for name in OPTIONAL:
        report[name] = version_of(name)
    for tool in ("ffmpeg", "ffprobe"):
        report[tool] = tool_version(tool)
        if report[tool] is None:
```

```
            failed.append(tool)
    try:
        import torch
        report["cuda"] = bool(torch.cuda.is_available())
    except Exception:  # noqa: BLE001
        report["cuda"] = False
    report["timestamp"] = datetime.now().isoformat(timespec="seconds")

    out = Path("results") / "environment_report.json"
    out.parent.mkdir(exist_ok=True)
    out.write_text(json.dumps(report, indent=2), encoding="utf-8")
    print(json.dumps(report, indent=2))
    if failed:
        print("FAILED:", ", ".join(failed), file=sys.stderr)
        return 1
    print("Environment OK")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```